

WEEK 03 MODULE
Programming Practice

TOPIC : 1 MERGE SORT

b) Medium

que. 2 **What is Sorting?**

- Sorting is the technique of arranging data elements in a particular order according to a chosen criterion, such as numerical value, alphabetical order, or date.

Types of Sorting :-

- i. Bubble Sort
- ii. Selection Sort
- iii. Insertion Sort
- iv. Merge Sort
- v. Quick Sort
- vi. Heap Sort



Merge Sort

Definition : Merge Sort is a comparison-based, divide-and-conquer sorting algorithm that sorts an array by repeatedly dividing it into smaller subarrays, sorting those subarrays, and then merging them back together in sorted order. Instead of sorting the entire array at once, Merge Sort breaks the problem into smaller, easier-to-solve parts, solves each part recursively, and combines the results to obtain the final sorted array.

Key Points:-

- Uses Divide and Conquer.
- Recursively splits the array.
- Merges sorted subarrays.
- Stable sorting algorithm.

How It Works:-

[38, 27, 43, 3]

↓ Divide

[38, 27] [43, 3]

↓ Divide

[38] [27] [43] [3]

↓ Merge

[27, 38] [3, 43]

↓ Final Merge

[3, 27, 38, 43]

Problem Statement

A large e-commerce company tracks the time taken (in minutes) to process customer orders before shipment. Due to varying warehouse workloads, processing times differ significantly from one order to another.

The analytics team wants to study operational efficiency by sorting all processing times and generating statistical information. Since millions of orders are processed every month, the company requires an efficient sorting technique with predictable performance.

Your task is to use **Merge Sort** to arrange all processing times in ascending order.

After sorting, perform the following operations:

1. Display the sorted processing times.
2. Find the **median processing time**.
3. Count the number of orders whose processing time is greater than the median.
4. Display the difference between the fastest and slowest processing time.

The solution should efficiently handle large datasets and demonstrate a clear understanding of Merge Sort's divide-and-conquer strategy

1. Question Analysis

A large e-commerce company records the processing time required to prepare customer orders before shipment. Since orders are received from different warehouses, the processing times are stored in random order and vary significantly depending on warehouse workload. To analyze operational efficiency and generate statistical information, the analytics team requires the processing times to be arranged in ascending order. After sorting, the median processing time, the number of orders whose processing time is greater than the median, and the difference between the fastest and slowest processing times must also be determined. As millions of orders are processed every month, an efficient sorting algorithm is required. Therefore, the task is to implement the **Merge Sort** algorithm to sort the processing times in ascending order without using any built-in sorting methods.

2. Problem Analysis

The given problem requires sorting an unsorted list of customer order processing times in ascending order using the Merge Sort algorithm. Since the e-commerce company processes millions of orders, the sorting technique should efficiently handle large datasets while maintaining consistent performance. Merge Sort solves this problem by recursively dividing the array into smaller subarrays until each subarray contains only one element. The sorted subarrays are then merged in ascending order to produce a completely sorted array. Once sorting is completed, the median processing time is determined, the number of orders having processing times greater than the median is counted, and the difference between the maximum and minimum processing times is calculated to generate the required statistical information.

3. Algorithm Selection Analysis

Selected Algorithm: Merge Sort

- Merge Sort follows the Divide and Conquer technique.
- It recursively divides the array into smaller subarrays and merges them in sorted order.
- It guarantees a time complexity of $O(N \log N)$ in the best, average, and worst cases.
- It efficiently handles large datasets containing millions of processing time records.
- The sorted array makes it easy to determine the median, count the orders above the median, and calculate the difference between the maximum and minimum processing times.
- Hence, Merge Sort is the most appropriate algorithm for solving this problem.

4. Points to Remember

- Merge Sort follows the Divide and Conquer technique.
- The array is recursively divided until each subarray contains only one element.
- Single-element subarrays are considered already sorted.
- Sorted subarrays are merged to produce larger sorted subarrays.

- The merging process continues until the complete array is sorted.
- After sorting, the median processing time is calculated.
- The number of orders having processing times greater than the median is counted.
- The difference between the fastest and slowest processing times is calculated using the minimum and maximum values of the sorted array.
- Time Complexity
 - Best Case : $O(N \log N)$
 - Average Case : $O(N \log N)$
 - Worst Case : $O(N \log N)$
- Space Complexity : $O(N)$

5. Core Idea / Logic

- Divide the given array of processing times into two equal halves.
- Continue dividing each subarray recursively until every subarray contains only one element.
- Consider each single element as an already sorted subarray.
- Merge two adjacent sorted subarrays into a larger sorted subarray in ascending order.
- Repeat the merging process until the entire array becomes sorted.
- Determine the median processing time from the sorted array.
- Count the number of processing times greater than the median.
- Calculate the difference between the maximum and minimum processing times.
- Display the sorted processing times along with all the required statistical information.

Part B : Steps

Step 1: Start the execution of the program.

Step 2: Read the number of customer orders (N) from the user.

Step 3: Validate the value of N. If N is less than 1 or greater than 100000, display an error message and terminate the program.

Step 4: Create an integer array of size N to store the processing times.

Step 5: Read the processing time of each customer order, validate that each value lies between 1 and 100000, and store the valid values in the array.

Step 6: Call the mergeSort(arr, 0, n - 1) method to sort the processing times.

Step 7: Recursively divide the array into smaller subarrays until each subarray contains only one element.

Step 8: Merge the divided subarrays in ascending order using the merge() method until the complete array becomes sorted.

Step 9: Display the sorted processing times.

Step 10: Find the median processing time by selecting the middle element of the sorted array.

Step 11: Traverse the sorted array and count the number of processing times that are greater than the median.

Step 12: Calculate the difference between the maximum and minimum processing times by subtracting the first element from the last element of the sorted array.

Step 13: Display the median processing time, the number of orders above the median, and the difference between the fastest and slowest processing times.

Step 14: Close the Scanner object and terminate the program.

7. Program Structure Analysis

i. Variable n

```
int n;
```

Purpose:

- Stores the total number of customer orders entered by the user.
- Determines the size of the array.
- Defines the number of processing times to be sorted.

- **Constraint:**

$$1 \leq n \leq 100000$$

In simple words:

- Minimum value of n can be 1.
- Maximum value of n can be 100000.

ii. Array arr[]

```
int[] arr = new int[n];
```

Purpose:

- Stores all customer order processing times.
- Acts as the main array on which Merge Sort is performed.
- Contains the sorted processing times after sorting is completed.

- **Constraint:**

$$1 \leq \text{Time} \leq 100000$$

In simple words:

- Minimum processing time can be 1 minute.
- Maximum processing time can be 100000 minutes.

iii. Temporary Arrays leftArray[] and rightArray[]

```
int[] leftArray = new int[n1];  
int[] rightArray = new int[n2];
```

Purpose:

- Temporarily store the left and right halves of the original array.
- Allow the merge() method to compare and merge elements efficiently.
- Preserve the original data while merging the sorted subarrays.

iv. Variables left, mid and right

```
mergeSort(arr, left, right);
```

Purpose:

- left stores the starting index of the current subarray.
- right stores the ending index of the current subarray.
- mid stores the middle index used to divide the array into two halves.
- Together, these variables control the recursive division of the array.

v. Variables i, j and k

```
int i = 0;
int j = 0;
int k = left;
```

Purpose:

- i traverses the leftArray.
- j traverses the rightArray.
- k stores the position in the original array where the next smallest element will be placed during merging.

vi. Variable median

```
int median = arr[n / 2];
```

Purpose:

- Stores the median processing time after the array has been sorted.
- Represents the middle element of the sorted array.
- Used as the reference value for counting orders above the median.

vii. Variable count

```
int count = 0;
```

Purpose:

- Counts the number of customer orders whose processing time is greater than the median.
- Increases whenever a processing time exceeds the median value.

viii. Variable difference

```
int difference = arr[n - 1] - arr[0];
```

Purpose:

- Stores the difference between the slowest and fastest processing times.
- Uses the last element (maximum) and first element (minimum) of the sorted array.
- Indicates the overall variation in processing times.

ix. Input Loop

```
for(int i = 0; i < n; i++)
{
    arr[i] = sc.nextInt();
}
```

Purpose:

- Accepts all processing times from the user.

- Validates each processing time according to the given constraints.
- Stores all valid processing times in the array.

x. mergeSort() Method

`mergeSort(arr, left, right);`

Purpose:

- Recursively divides the array into smaller subarrays.
- Continues dividing until each subarray contains only one element.
- Calls the `merge()` method to combine the sorted subarrays.
- Produces the final sorted array.

xi. merge() Method

`merge(arr, left, mid, right);`

Purpose:

- Combines two sorted subarrays into one larger sorted subarray.
- Compares elements from both temporary arrays.
- Places the smaller element into the original array.
- Continues until all elements are merged in ascending order.

xii. Merge Comparison Loop

`while(i < n1 && j < n2)`

Purpose:

- Compares the current elements of the left and right temporary arrays.
- Places the smaller element into the original array.
- Continues until one of the temporary arrays is completely traversed.

xiii. Remaining Element Loops

`while(i < n1)`
`while(j < n2)`

Purpose:

- Copy any remaining elements from the left temporary array.
- Copy any remaining elements from the right temporary array.
- Ensure that all elements are transferred back to the original array after merging.

xiv. Count Loop

`for(int value : arr)`

Purpose:

- Traverses the sorted array.
- Checks whether each processing time is greater than the median.
- Increments the count variable whenever the condition is satisfied.

xv. Output Loop

```
for(int i = 0; i < n; i++)
{
    System.out.print(arr[i] + " ");
}
```

Purpose:

- Traverses the sorted array.
- Displays all processing times in ascending order.
- Produces the final sorted output before displaying the calculated statistics.

8. program

```
package week3_module;
import java.util.Scanner;
public class MergeSortMedium {

    // Function to merge two sorted subarrays
    static void merge(int[] arr, int left, int mid, int right) {

        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] leftArray = new int[n1];
        int[] rightArray = new int[n2];

        // Copy elements into temporary arrays
        for (int i = 0; i < n1; i++) {
            leftArray[i] = arr[left + i];
        }

        for (int j = 0; j < n2; j++) {
            rightArray[j] = arr[mid + 1 + j];
        }

        int i = 0;
        int j = 0;
        int k = left;

        // Merge the temporary arrays
        while (i < n1 && j < n2) {

            if (leftArray[i] <= rightArray[j]) {
                arr[k] = leftArray[i];
                i++;
            } else {
                arr[k] = rightArray[j];
                j++;
            }

            k++;
        }

        // Copy remaining elements of leftArray
        while (i < n1) {

            arr[k] = leftArray[i];
```

```

        i++;
        k++;
    }

    // Copy remaining elements of rightArray
    while (j < n2) {
        arr[k] = rightArray[j];
        j++;
        k++;
    }
}

// Recursive Merge Sort function
static void mergeSort(int[] arr, int left, int right) {

    if (left < right) {

        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter the number of customer orders : ");
    int n = sc.nextInt();

    if (n < 1 || n > 100000) {
        System.out.println("Error: N must be between 1 and 100000.");
        sc.close();
        return;
    }

    int[] arr = new int[n];

    System.out.print("Enter the processing times (in minutes) : ");
    for (int i = 0; i < n; i++) {

        arr[i] = sc.nextInt();

        if (arr[i] < 1 || arr[i] > 100000) {
            System.out.println("Error: Processing time must be between 1 and 100000.");
            sc.close();
            return;
        }
    }

    // Perform Merge Sort
    mergeSort(arr, 0, n - 1);

    // Display sorted processing times
    System.out.print("Sorted Processing Times : ");
    for (int i = 0; i < n; i++) {
        System.out.print(arr[i] + " ");
    }

    System.out.println();
}

```



```

// Find median
int median = arr[n / 2];
System.out.println("Median: " + median);

// Count orders above median
int count = 0;
for (int value : arr) {
    if (value > median) {
        count++;
    }
}
System.out.println("Orders Above Median: " + count);

// Find difference
int difference = arr[n - 1] - arr[0];
System.out.println("Difference: " + difference);

sc.close();
}
}

```

9. Program's output

Enter the number of customer orders : 7

Enter the processing times (in minutes) : 12 25 18 30 15 22 40

Sorted Processing Times : 12 15 18 22 25 30 40

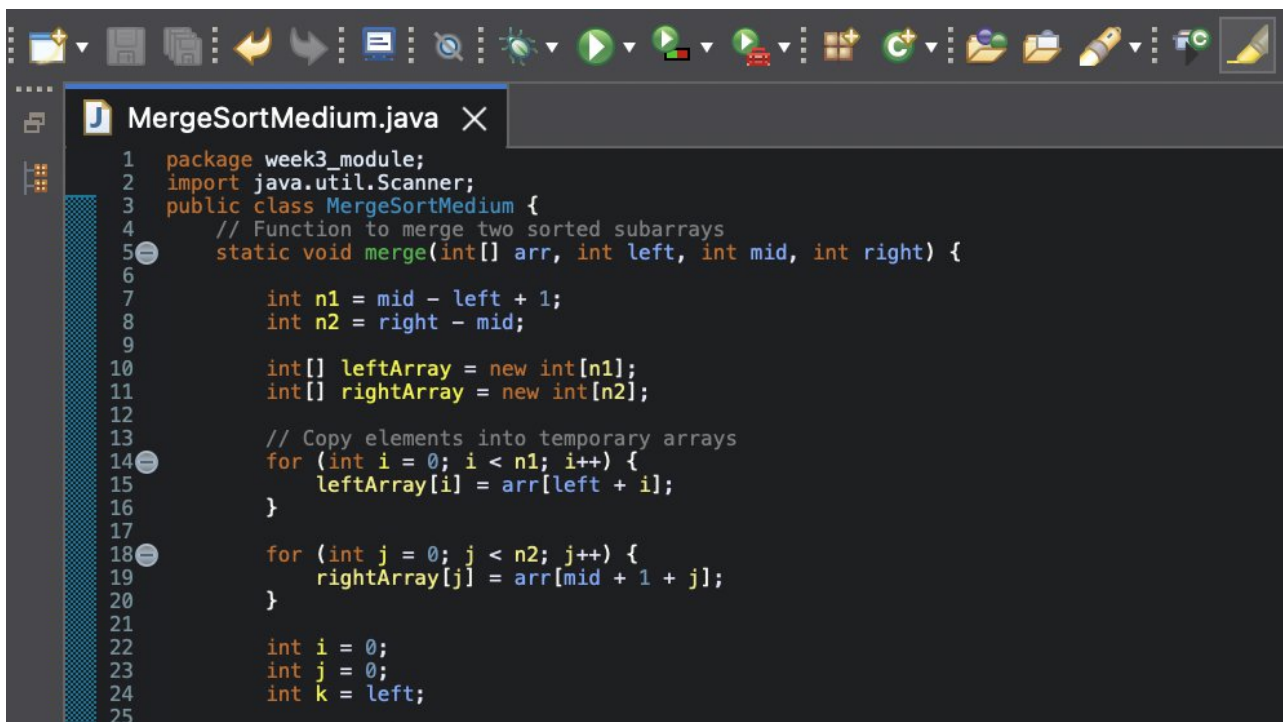
Median: 22

Orders Above Median: 3

Difference: 28

10. screenshots

program



```

MergeSortMedium.java
1 package week3_module;
2 import java.util.Scanner;
3 public class MergeSortMedium {
4     // Function to merge two sorted subarrays
5     static void merge(int[] arr, int left, int mid, int right) {
6
7         int n1 = mid - left + 1;
8         int n2 = right - mid;
9
10        int[] leftArray = new int[n1];
11        int[] rightArray = new int[n2];
12
13        // Copy elements into temporary arrays
14        for (int i = 0; i < n1; i++) {
15            leftArray[i] = arr[left + i];
16        }
17
18        for (int j = 0; j < n2; j++) {
19            rightArray[j] = arr[mid + 1 + j];
20        }
21
22        int i = 0;
23        int j = 0;
24        int k = left;
25
26        // Merge the temporary arrays
27        while (i < n1 && j < n2) {
28
29            if (leftArray[i] <= rightArray[j]) {
30                arr[k] = leftArray[i];
31                i++;
32            } else {

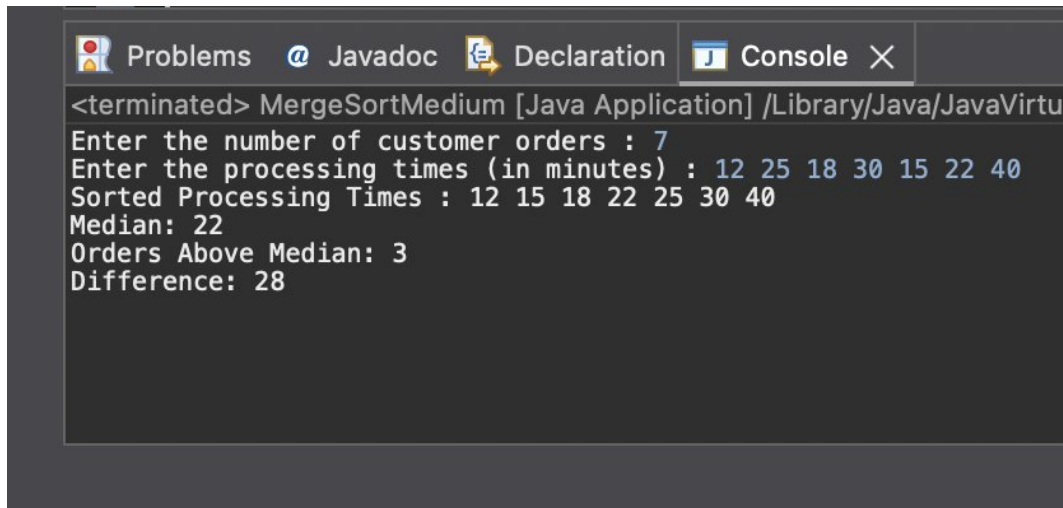
```

```

73     System.out.print("Enter the number of customer orders : ");
74     int n = sc.nextInt();
75
76     if (n < 1 || n > 100000) {
77         System.out.println("Error: N must be between 1 and 100000.");
78         sc.close();
79         return;
80     }
81
82     int[] arr = new int[n];
83
84     System.out.print("Enter the processing times (in minutes) : ");
85     for (int i = 0; i < n; i++) {
86
87         arr[i] = sc.nextInt();
88
89         if (arr[i] < 1 || arr[i] > 100000) {
90             System.out.println("Error: Processing time must be between 1 and 100000.");
91             sc.close();
92             return;
93         }
94     }
95
96     // Perform Merge Sort
97     mergeSort(arr, 0, n - 1);
98
99     // Display sorted processing times
100    System.out.print("Sorted Processing Times : ");
101    for (int i = 0; i < n; i++) {
102        System.out.print(arr[i] + " ");
103    }
104    System.out.println();
105
106    // Find median
107    int median = arr[n / 2];
108    System.out.println("Median: " + median);
109
110    // Count orders above median
111    int count = 0;
112    for (int value : arr) {
113        if (value > median) {
114            count++;
115        }
116    }
117    System.out.println("Orders Above Median: " + count);
118
119    // Find difference
120    int difference = arr[n - 1] - arr[0];
121    System.out.println("Difference: " + difference);
122
123    sc.close();
124 }
125 }

```

program's output



```
<terminated> MergeSortMedium [Java Application] /Library/Java/JavaVirtu
Enter the number of customer orders : 7
Enter the processing times (in minutes) : 12 25 18 30 15 22 40
Sorted Processing Times : 12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
```

11. EXPLANATION :-

Each order takes some time to process.

Example: 12 25 18 30 15 22 40

These are processing times (in minutes).

They're in random order.

But now you have to :

- Sort them from smallest to largest.
- find the median,
- how many orders took longer than the median,
- and the difference between the fastest and slowest order.

That's exactly what our program does.

Step 1 – main() starts

Every Java program begins here.

```
public static void main(String[] args)
```

Step 2 – Scanner

```
Scanner sc = new Scanner(System.in);
```

Step 3 – Read N

```
int n = sc.nextInt();
```

Suppose n = 7

Meaning there are 7 processing times to be entered.

Step 4 – Validate N

```
if(n<1 || n>100000)
```

Purpose:

- Checks whether the entered package satisfies the given constraints.
- Displays an error message if an invalid package no is entered.
- Terminates the program to prevent invalid data from being processed.

Step 5 – Create Array

```
int arr[] = new int[n];
```

Suppose n=7

Java creates

Step 6 – Input Loop

```
for(int i=0;i<n;i++)  
arr[i]=sc.nextInt();
```

Suppose user enters 12

Stored here :- 12 -----

Next 25

Stored :- 12 25 -----

Then 18

Stored :- 12 25 18 ----

Eventually :- 12 25 18 30 15 22 40

Step 7 – Validate Each Processing Time

Immediately after reading `arr[i]=sc.nextInt();`

we check `if(arr[i]<1 || arr[i]>100000)`

Question says

$1 \leq \text{Time} \leq 100000$

Step 8 – Call Merge Sort

```
mergeSort(arr,0,n-1);
```

Indexes :- 0 1 2 3 4 5 6

Current array :- 12 25 18 30 15 22 40

Control now leaves main() and enters mergeSort()

Step 9 – mergeSort()

```
mergeSort(arr,left,right)
```

Initially left=0 & right=6

Question: Can we divide this array?

Yes.

First mid = $0+(6-0)/2 = 3$

Now array is divided into

- 12 25 18 30

- 15 22 40

Merge Sort doesn't sort yet. It only divides.

Again

Left part :- 12 25 18 30

becomes

- 12 25

- 18 30

Again

- 12

- 25

Eventually everything becomes :

12

25

18

30

15

22

a single element is already sorted. There is nothing to compare.

Step 10 – merge()

First 12 & 25

Merge : 12 25 = Already sorted.

Then 18 & 30

Merge : 18 30 = Already sorted.

Now

merge 12 25 & 18 30

Result :- 12 18 25 30

This is why Merge Sort is called Merge Sort. It sorts while merging.

Same happens

on the right side.

becomes :- 15 22 40

- Left :- 12 18 25 30
- Right :- 15 22 40
- Finally :- 12 15 18 22 25 30 40

Now the array is completely sorted.

Control returns to main()

Step 11 – Print Sorted Array

```
for(int i=0;i<n;i++)
```

prints :- 12 15 18 22 25 30 40

Step 12 – Find Median

```
int median=arr[n/2];
```

Question : What is median?

Median means the middle element of a sorted array.

Indexes :- 0 1 2 3 4 5 6

sorted array :- 12 15 18 22 25 30 40

Now $n=7$

So, $n/2 = 7/2 = 3$

Therefore $\text{arr}[3] = 22$

So, Median=22

Step 13 – Count Orders Above Median

`int count=0;`

Initially `count=0`

Loop :- `for(int value:arr)`

$12 > 22 ? = \text{No}$

Count = 0

Next $15 > 22 ? = \text{No}$

Next $18 > 22 ? = \text{No}$

Next $22 > 22 ? = \text{No}$

Next $25 > 22 ? = \text{Yes}$

Count = 1

Next $30 > 22 ? = \text{Yes}$

Count = 2

Next $40 > 22 ? = \text{Yes}$

Count = 3

Final answer :- Orders Above Median = 3

Step 14 – Difference

`int difference=arr[n-1]-arr[0];`

Question

`arr[0]` Because after sorting the first element is always the **smallest**.

`arr[n-1]` Because the last element is always the **largest**.

40

Difference :- $40-12= 28$

Step 15 – Print Everything

Program prints :- Sorted Processing Times = 12 15 18 22 25 30 40

Median :22

Orders Above Median :3

Difference :28

Step 16 – Close Scanner

```
sc.close();
```

The program has finished taking input, so the Scanner is closed to free system resources.

Overall Program Flow

Start



Read N



Validate N



Create Array



Read Processing Times



Validate Each Processing Time



Call Merge Sort



Recursively Divide the Array



Merge the Subarrays in Sorted Order



Display the Sorted Processing Times



Find the Median



Count Orders Above the Median



Calculate the Difference (Maximum — Minimum)



Display All Results



Close Scanner



End Program

The one concept to remember

The **sorting** and the **analysis** are two separate stages:

- **Stage 1:** Use **Merge Sort** to transform an unsorted array into a sorted one.
- **Stage 2:** Once the array is sorted:
 - The **median** is easy to find because it's the middle element.
 - Counting **orders above the median** is just a simple traversal.
 - The **difference** is simply the last element (maximum) minus the first element (minimum).